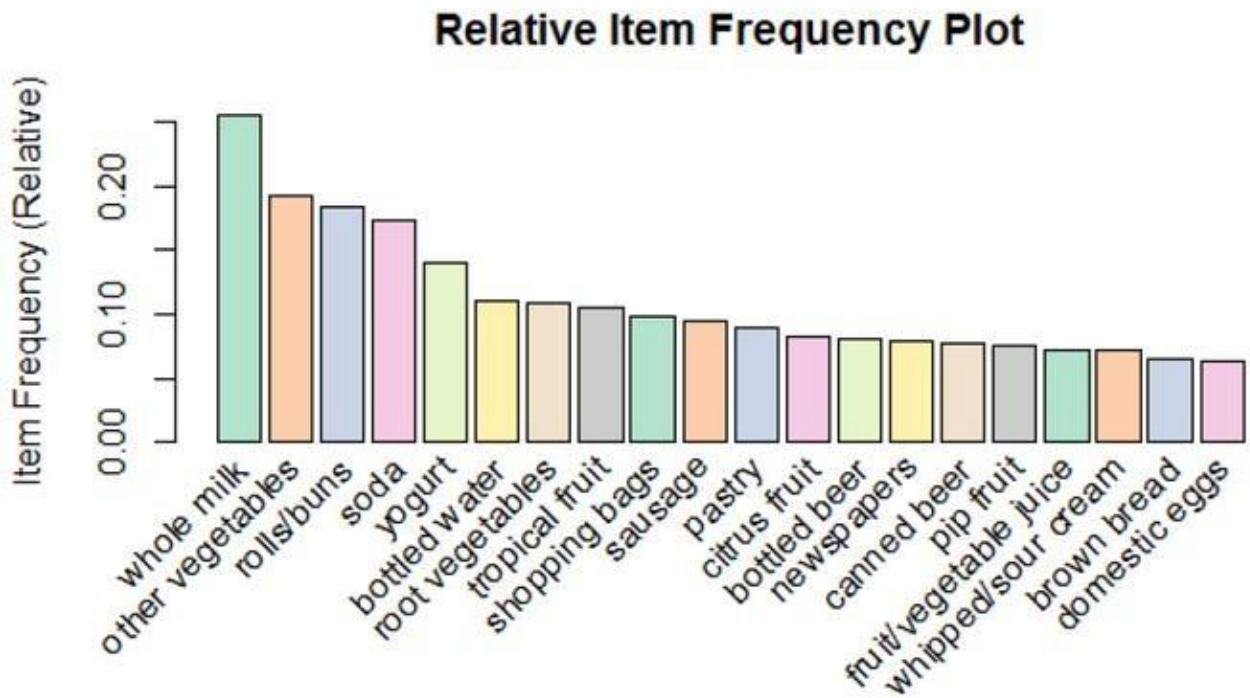
**PROGRAM:**

```
# Loading Libraries
library(arules)
library(arulesViz)
library(RColorBrewer)

# import dataset
data("Groceries")

# using apriori() function
rules <- apriori(Groceries,
                 parameter = list(supp = 0.01, conf = 0.2))

# using inspect() function
inspect(rules[1:10])

# using itemFrequencyPlot() function
arules::itemFrequencyPlot(Groceries, topN = 20,
                          col = brewer.pal(8, 'Pastel2'),
                          main = 'Relative Item FrequencyPlot',
                          type = "relative",
                          ylab = "Item Frequency (Relative)")
```

**OUTPUT:**

```
Console   Terminal
~/
> rules <- apriori(Groceries,parameter = list(supp = 0.01, conf = 0.2))
Apriori

Parameter specification:
 confidence minval smax arem  aval originalSupport maxtime support minlen maxlen
        0.2    0.1    1 none FALSE            TRUE       5    0.01      1     10
 target   ext
  rules FALSE

Algorithmic control:
 filter tree heap memopt load sort verbose
    0.1 TRUE TRUE  FALSE TRUE    2    TRUE

Absolute minimum support count: 98

set item appearances ...[0 item(s)] done [0.00s].
set transactions ...[169 item(s), 9835 transaction(s)] done [0.01s].
sorting and recoding items ... [88 item(s)] done [0.00s].
creating transaction tree ... done [0.01s].
checking subsets of size 1 2 3 4 done [0.01s].
writing ... [232 rule(s)] done [0.00s].
creating S4 object  ... done [0.01s].
> inspect(rules[1:10])
     lhs                rhs                support    confidence lift     count
[1]  {}              => {whole milk}       0.25551601 0.2555160  1.000000 2513
[2]  {hard cheese}   => {whole milk}       0.01006609 0.4107884  1.607682   99
[3]  {butter milk}   => {other vegetables} 0.01037112 0.3709091  1.916916  102
[4]  {butter milk}   => {whole milk}       0.01159126 0.4145455  1.622385  114
[5]  {ham}           => {whole milk}       0.01148958 0.4414062  1.727509  113
[6]  {sliced cheese} => {whole milk}       0.01077783 0.4398340  1.721356  106
[7]  {oil}           => {whole milk}       0.01128622 0.4021739  1.573968  111
[8]  {onions}        => {other vegetables} 0.01423488 0.4590164  2.372268  140
[9]  {onions}        => {whole milk}       0.01209964 0.3901639  1.526965  119
[10] {berries}       => {yogurt}           0.01057448 0.3180428  2.279848  104
> arules::itemFrequencyPlot(Groceries,topN=20,col=brewer.pal(8,'Pastel2'),main='Relativ
e Item Frequency Plot',type="relative",ylab="Item Frequency (Relative)")
```



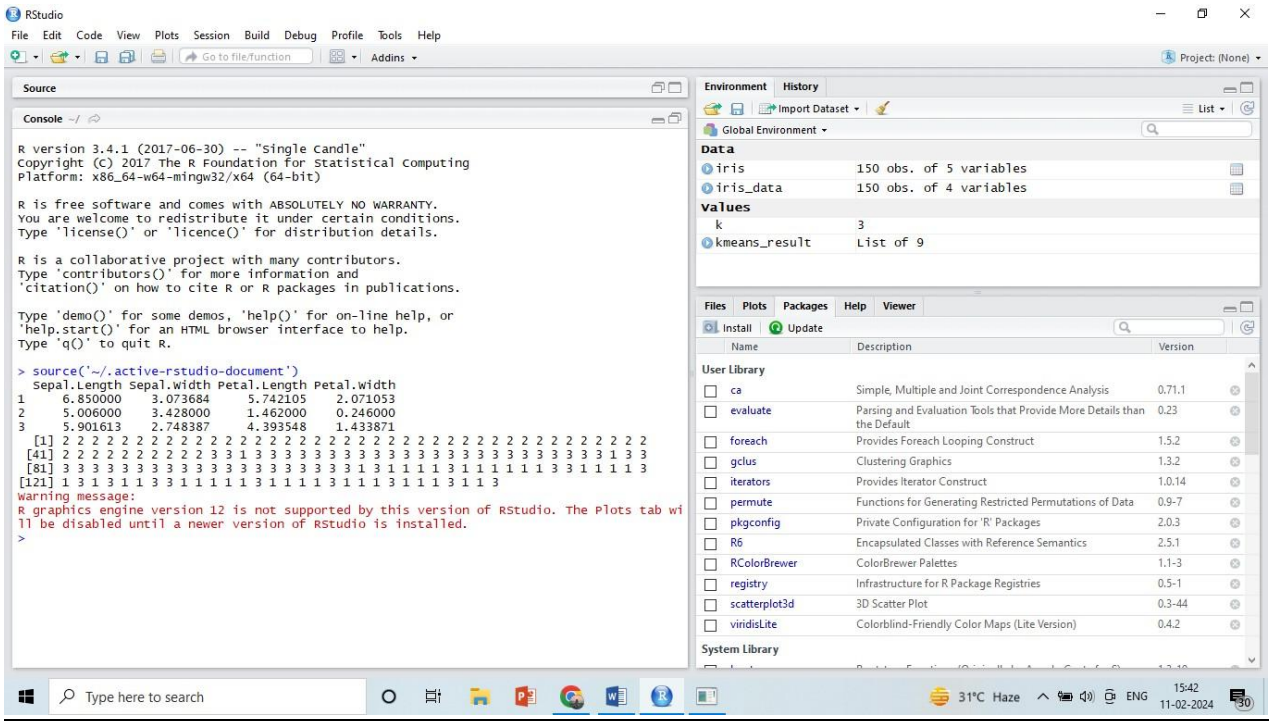
**RESULT:**

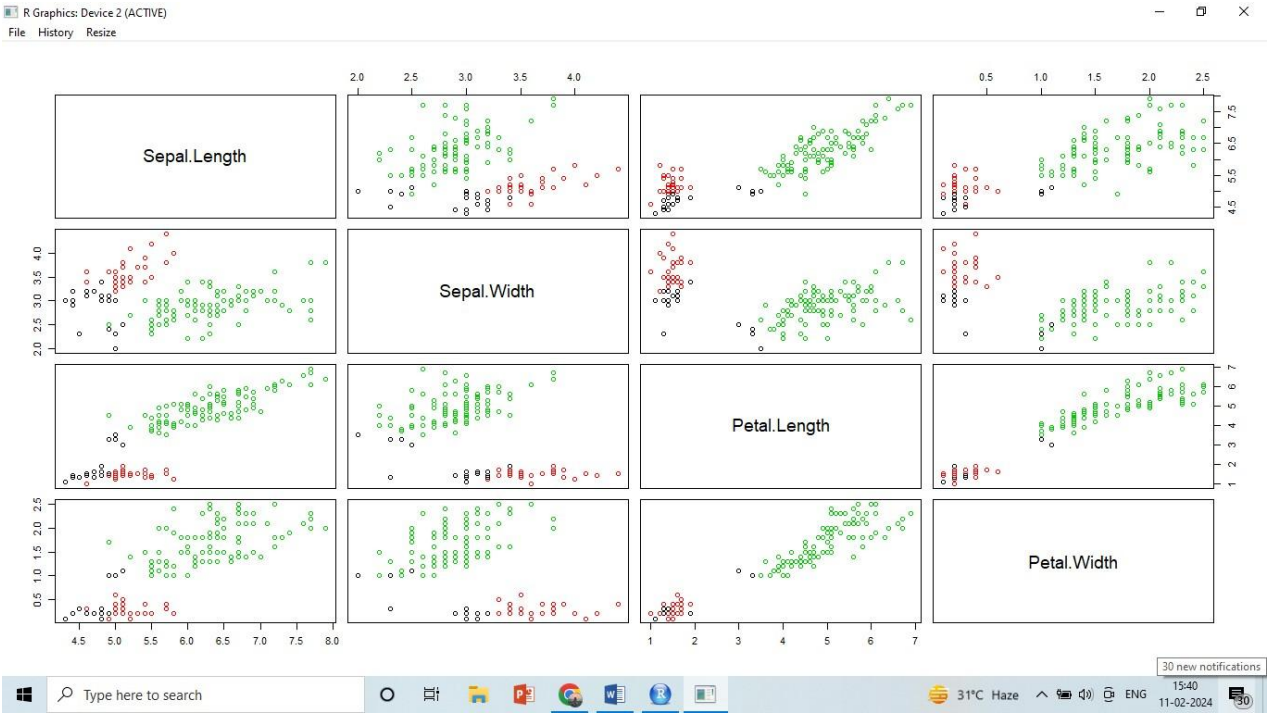
Thus the above program has been created and executed successfully

**PROGRAM:**

```
# Load required library
library(datasets)

# Load your dataset
data(iris)
iris_data <- iris[, -5]  # Exclude the species column

# Perform k-means clustering
k <- 3  # Number of clusters
kmeans_result <- kmeans(iris_data, centers = k)

# View the cluster centers
print(kmeans_result$centers)

# View the cluster assignments for each data point
print(kmeans_result$cluster)

# Plot the data points with cluster assignments
 plot(iris_data, col = kmeans_result$cluster)
points(kmeans_result$centers, col = 1:k, pch = 8, cex = 2)
```

## OUTPUT:


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source
Console ~/
R version 3.4.1 (2017-06-30) -- "Single Candle"
Copyright (C) 2017 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
> source('~/.active-rstudio-document')
Sepal.Length Sepal.Width Petal.Length Petal.Width
1 6.850000 3.073684 5.742105 2.071053
2 5.006000 3.428000 1.462000 0.246000
3 5.901613 2.748387 4.393548 1.433871
warning message:
R graphics engine version 12 is not supported by this version of RStudio. The Plots tab will be disabled until a newer version of RStudio is installed.
Environment History
Import Dataset
Global Environment
Data
iris 150 obs. of 5 variables
iris_data 150 obs. of 4 variables
values
k 3
kmeans_result List of 9
Files Plots Packages Help Viewer
Install Update
Name Description Version
User Library
ca Simple, Multiple and Joint Correspondence Analysis 0.71.1
evaluate Parsing and Evaluation Tools that Provide More Details than the Default 0.23
foreach Provides Foreach Looping Construct 1.5.2
gclus Clustering Graphics 1.3.2
iterators Provides Iterator Construct 1.0.14
permute Functions for Generating Restricted Permutations of Data 0.9-7
pkgconfig Private Configuration for 'R' Packages 2.0.3
R6 Encapsulated Classes with Reference Semantics 2.5.1
RColorBrewer ColorBrewer Palettes 1.1-3
registry Infrastructure for R Package Registries 0.5-1
scatterplot3d 3D Scatter Plot 0.3-44
viridisLite Colorblind-Friendly Color Maps (Lite Version) 0.4.2
System Library
Type here to search
31°C Haze
ENG
15:42
11-02-2024


R Graphics: Device 2 (ACTIVE)
File History Resize
Sepal.Length
Sepal.Width
Petal.Length
Petal.Width
Type here to search
30 new notifications
31°C Haze
ENG
15:40
11-02-2024

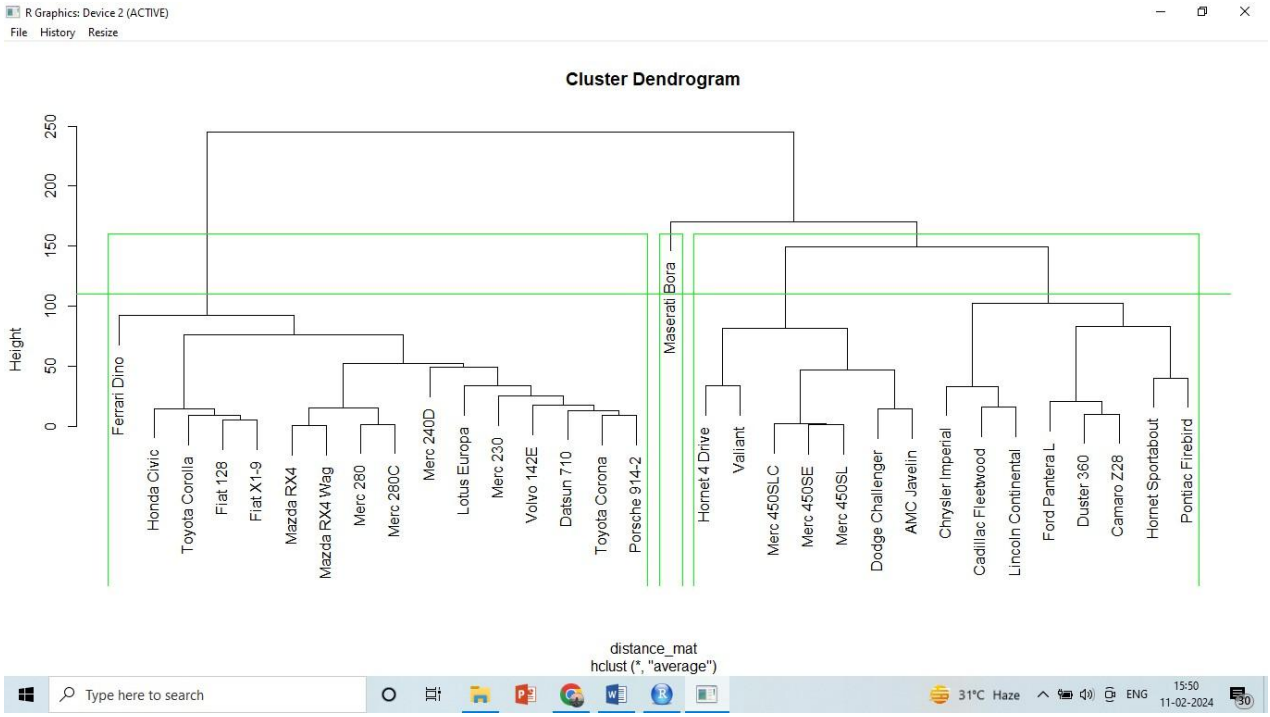
**RESULT:**

Thus the above program has been created and executed successfully

**PROGRAM:**

```
# Finding distance matrix
distance_mat <- dist(mtcars, method = 'euclidean')
distance_mat

# Fitting Hierarchical clustering Model
# to training dataset
set.seed(240) # Setting seed
Hierar_cl <- hclust(distance_mat, method = "average")
Hierar_cl

# Plotting dendrogram
plot(Hierar_cl)

# Choosing no. of clusters #
Cutting tree by height
abline(h = 110, col = "green")

# Cutting tree by no. of clusters fit
<- cutree(Hierar_cl, k = 3 )
fit

table(fit)
rect.hclust(Hierar_cl, k = 3, border = "green")
```

## OUTPUT:




## RESULT:

Thus the above program has been created and executed successfully

**PROGRAM:**

```
# Installing Packages
install.packages("e1071")
install.packages("caTools")
install.packages("caret")

# Loading package
library(e1071)
library(caTools)
library(caret)

# Splitting data into train
# and test data
split <- sample.split(iris, SplitRatio = 0.7)
train_cl <- subset(iris, split == "TRUE")
test_cl <- subset(iris, split == "FALSE")

# Feature Scaling
train_scale <- scale(train_cl[, 1:4])
test_scale <- scale(test_cl[, 1:4])

# Fitting Naive Bayes Model  #
to training dataset
set.seed(120) # Setting Seed
classifier_cl <- naiveBayes(Species ~ ., data = train_cl)
classifier_cl

# Predicting on test data'
y_pred <- predict(classifier_cl, newdata = test_cl)
```

```
# Confusion Matrix
cm <- table(test_cl$Species, y_pred)
cm

# Model Evaluation
confusionMatrix(cm)
```

**OUTPUT:**

```
> classifier_cl

Naive Bayes Classifier for Discrete Predictors

Call:
naiveBayes.default(x = X, y = Y, laplace = laplace)

A-priori probabilities:
Y
    setosa versicolor  virginica
 0.3333333  0.3333333  0.3333333

Conditional probabilities:
            Sepal.Length
Y                [,1]      [,2]
  setosa     4.946667 0.3664681
  versicolor 5.870000 0.5018243
  virginica  6.573333 0.5765135

            Sepal.Width
Y                [,1]      [,2]
  setosa     3.430000 0.3405371
  versicolor 2.676667 0.2582211
  virginica  2.946667 0.3308644

            Petal.Length
Y                [,1]      [,2]
  setosa     1.456667 0.1924136
  versicolor 4.196667 0.5235709
  virginica  5.590000 0.5498276

            Petal.Width
Y                 [,1]      [,2]
  setosa     0.2233333 0.1104328
  versicolor 1.2966667 0.1920548
  virginica  1.9900000 0.2904812
```

**RESULT:**

Thus the above program has been created and executed successfully

**PROGRAM:**

```
# Load the party package. It will automatically load other
# dependent packages.
library(party)

# Create the input data frame.
input.data <- readingSkills[c(1:105), ]

# Give the chart file a name.
png(file = "decision_tree.png")

# Create the tree.
output.tree <- ctree(
nativeSpeaker ~ age + shoeSize + score,
data = input.dat)

# Plot the tree.
plot(output.tree)

# Save the file.
dev.off()
```

**OUTPUT:**

```
null device
          1
Loading required package: methods
Loading required package: grid
Loading required package: mvtnorm
Loading required package: modeltools
Loading required package: stats4
Loading required package: strucchange
Loading required package: zoo

Attaching package: 'zoo'

The following objects are masked from 'package:base':

   as.Date, as.Date.numeric

Loading required package: sandwich
```

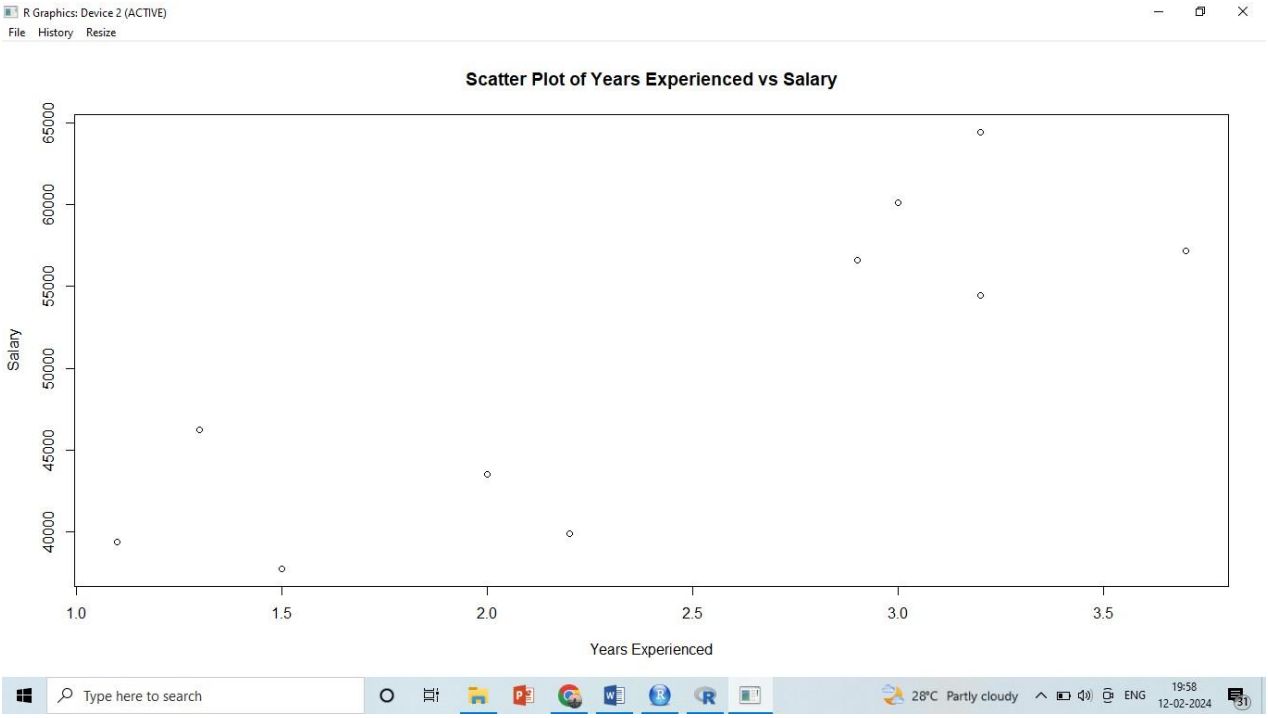
**RESULT:**

Thus the above program has been created and executed successfully

**PROGRAM:**

```
# Create the data frame data
<- data.frame(
Years_Exp = c(1.1, 1.3, 1.5, 2.0, 2.2, 2.9, 3.0, 3.2, 3.2, 3.7),
Salary = c(39343.00, 46205.00, 37731.00, 43525.00,
                                39891.00, 56642.00, 60150.00, 54445.00, 64445.00, 57189.00)
)

# Create the scatter plot
plot(data$Years_Exp, data$Salary,
        xlab = "Years Experienced",
        ylab = "Salary",
        main = "Scatter Plot of Years Experienced vs Salary")
```

**OUTPUT:**




**RESULT:**

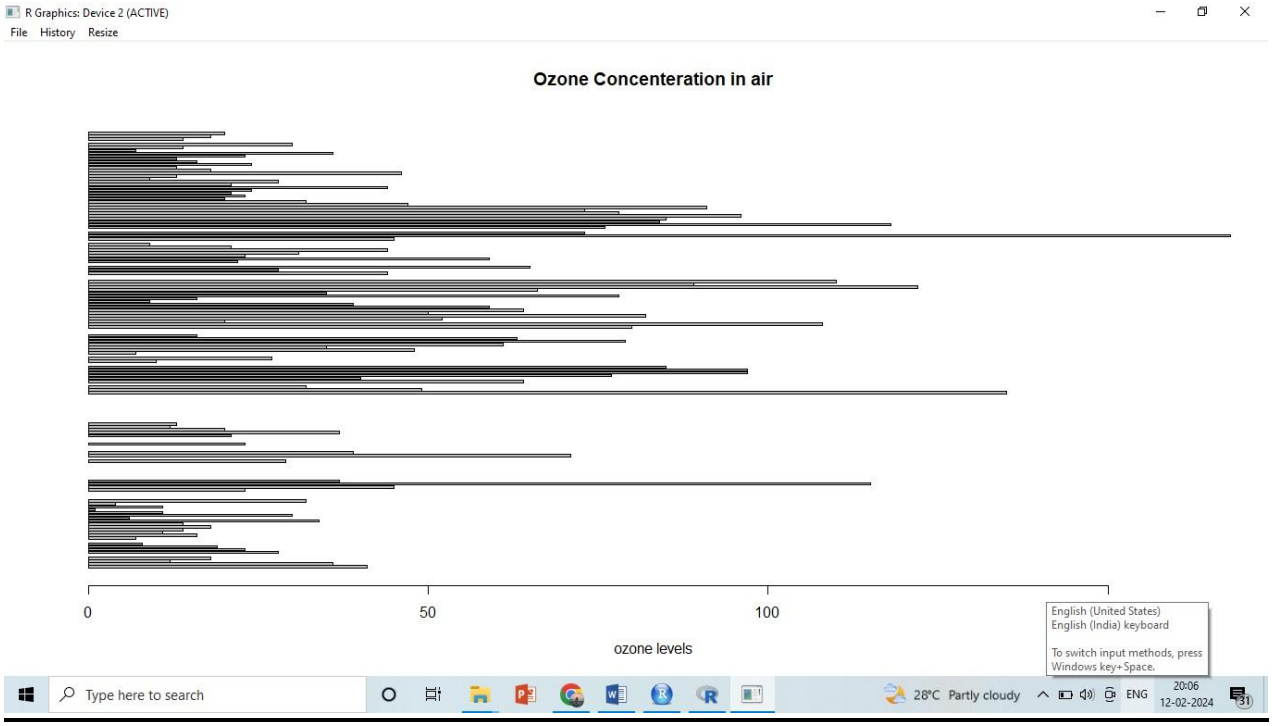
Thus the above program has been created and executed successfully

**PROGRAM:**

## Horizontal Bar Plot for

```
# Horizontal Bar Plot for
# Ozone concentration in air
barplot(airquality$Ozone,
                main = 'Ozone Concenteration in air',
                xlab = 'ozone levels', horiz = TRUE)
```

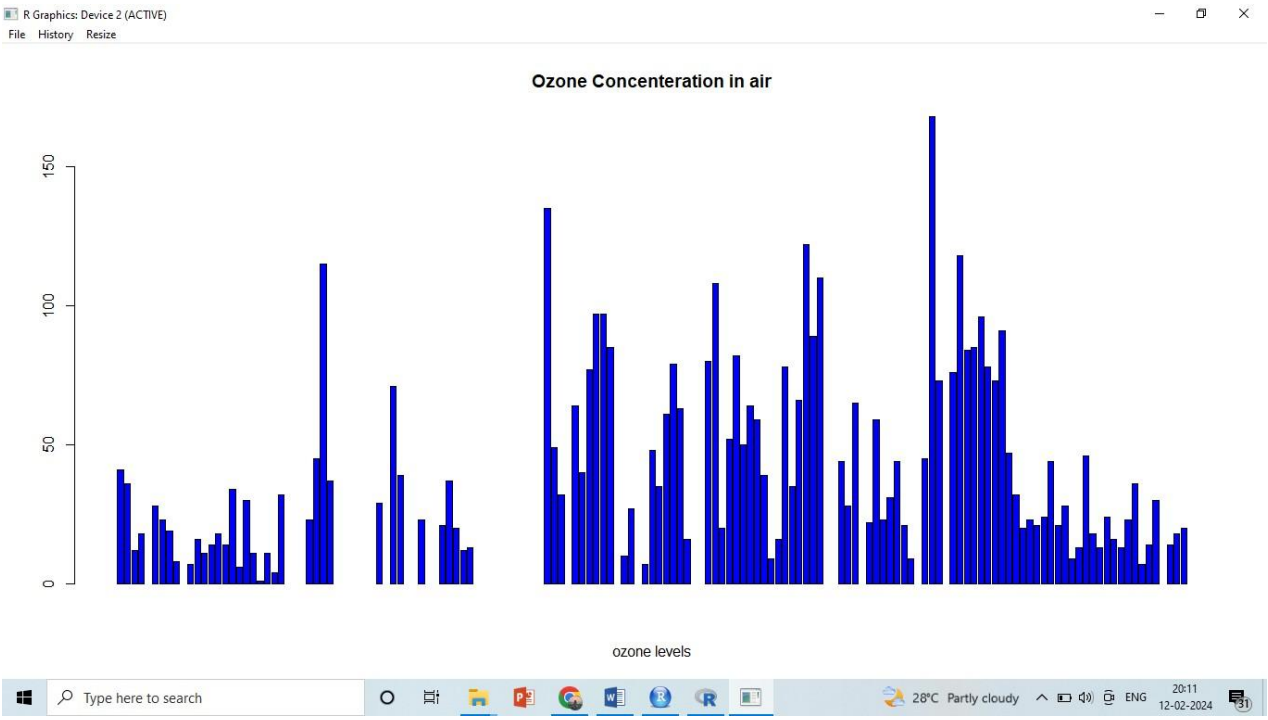
**OUTPUT**



## Vertical Bar Plot

```
# Vertical Bar Plot for
# Ozone concentration in air
barplot(airquality$Ozone, main = 'Ozone Concenteration in air',
        xlab = 'ozone levels', col ='blue', horiz = FALSE)
```

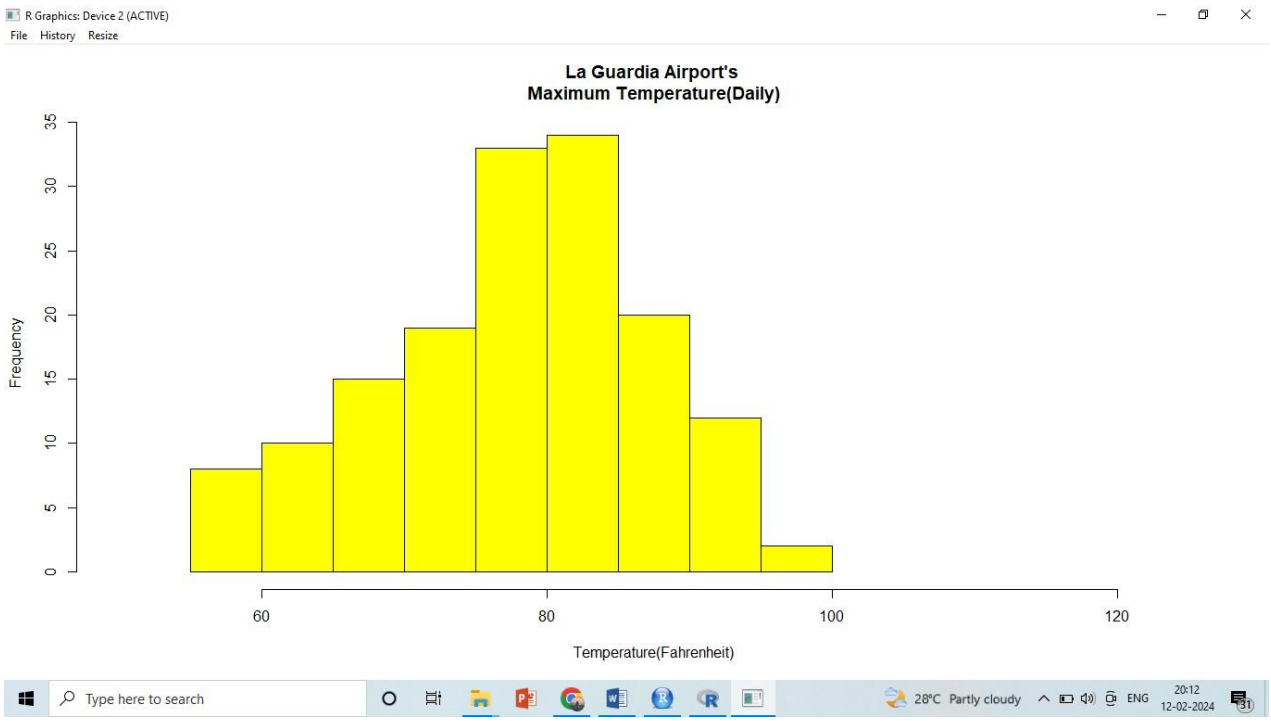
## <u>OUTPUT</u>



## Histogram

```
# Histogram for Maximum Daily Temperature
data(airquality)

hist(airquality$Temp, main ="La Guardia Airport's\
Maximum Temperature(Daily)",
        xlab ="Temperature(Fahrenheit)",
        xlim = c(50, 125), col ="yellow",
        freq = TRUE)
```

## <u>OUTPUT</u>



**RESULT:**

Thus the above program has been created and executed successfully